

Inteligencia Artificial

Episodio X
DeLP (Defeasible Logic Programming)

Tercera parte:

líneas de argumentación aceptable, árbol de dialéctica y respuestas a una consulta

Dr. ALEJANDRO JAVIER GARCÍA http://cs.uns.edu.ar/~ajg

DEPARTAMENTO DE CIENCIAS E INGENIERÍA DE LA COMPUTACIÓN - UNIVERSIDAD NACIONAL DEL SUR - BAHÍA BLANCA - ARGENTINA

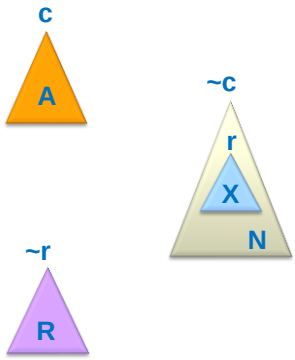
Notación gráfica de argumentos

Voy a usar una notación gráfica que será muy útil para indicar como se relacionan un conjunto de argumentos.

Como un argumento tiene una única conclusión, y requiere de una derivación que la sustente, entonces voy a usar un triángulo como la forma gráfica de un argumento.

La conclusión (generalmente abreviada) estará indicada en el vértice superior, y el nombre del conjunto de reglas (o nombre del argumento) dentro del triángulo.

Si un argumento es parte de otro argumento (sub-argumento) entonces un triángulo estará dentro del otro.



Algunos argumentos de (Π_m, Δ_m) sus abreviaturas y su notación gráfica

<A, c > = <A, comprar_acciones(acme) **>**

$$A = \left\{ \begin{array}{l} \text{comprar_acciones(acme)} \prec \text{buen_precio(acme).} \\ \text{buen_precio(acme)} \prec \text{buen_precio_web(acme).} \end{array} \right\}$$

<N, ~c> = <N, ~comprar_acciones(acme) **>**

$$N = \left\{ \begin{array}{l} \sim\text{comprar_acciones(acme)} \prec \text{riesgosa(acme).} \\ \text{riesgosa(acme)} \prec \text{en_fusion(acme,strong).} \end{array} \right\}$$

< X, r > = <X, riesgosa(acme) **>**

$$X = \left\{ \text{riesgosa(acme)} \prec \text{en_fusion(acme,strong).} \right\}$$

< R, ~r > = <**R,** ~riesgosa(acme) **>**

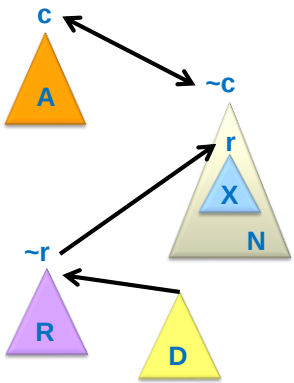
$$R = \left\{ \sim\text{riesgosa(acme)} \prec \text{en_fusion(acme,strong), fuerte(strong).} \right\}$$

Relaciones entre argumentos

Resumen de las relaciones que puede haber entre dos argumentos

Las siguientes relaciones pueden ocurrir entre dos argumentos de un PLR (luego veremos todos los detalles técnicos):

- Desacuerdo o conflicto
- Sub-argumento
- Ataque (combina desacuerdo y sub-argumento)
- Preferencia
- Derrota (combina ataque y preferencia)



Algunos "***spoilers***" sobre lo que exploraremos en detalle a continuación:

- El argumento **A** expresa una razón para la conclusión "c", y **N** para "~c"
- Los literales "~c" y "c" son conclusiones contradictorias y están en conflicto.
- Por lo tanto, **N** expresa una razón en contra de "c", y **A** en contra de "~c".
- Los argumentos **N** y **A** están en conflicto y se atacan mutuamente (**doble flecha**).
- Si **R** es preferido a **X**, hay una derrota propia de **R** a **X** (**flecha simple**).
- Todo ataque a un sub-argumento es un ataque al súper-argumento, por esa razón, como **R** está en conflicto con un punto interno a **N**, entonces también lo está con **N**.
- Como **R** derrota a **X**, y **X** es sub-argumento de **N** entonces **R** también derrota a **N.**
- Si **R** derrota a **N** entonces **R** indirectamente defiende a todo lo que **N** está atacando.
- Como **R** ataca a **N,** entonces **R** indirectamente defiende a **A.**
- Si hubiera un argumento **D** que ataque a **R,** entonces **D** indirectamente defendería a **N,** y por lo tanto **D** indirectamente atacaría a **A.**

Relaciones entre argumentos:

desacuerdo

Relaciones entre argumentos: desacuerdo

Dado un programa (Π, Δ) dos argumentos <X,f> y <Y,g> están en desacuerdo (o en conflicto) si $\{f,g\} \cup \Pi$ es un conjunto contradictorio (deriva dos literales complementarios).

A partir de (Π_2, Δ_2) se pueden construir < **A**,g> < **B**,f> <**C**,~g> y <**D**,e>

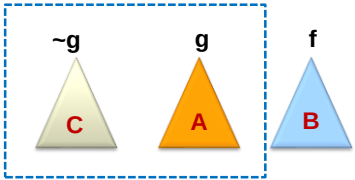
$\Pi_2 = \{ \sim p \leftarrow g.,\ p \leftarrow f.,\ b.,\ a. \}$ $\Delta_2 = \{ g -< a.,\ f -< b.,\ \sim g -< b.,\ e -< f. \}$

A = { g–< a. }

B = { f–< b. }

C = { ~g–< b. }

D = { e–< f, f–< b }



< **A**,g> y <**C**,~g> están en desacuerdo ya que a partir de $\Pi_2 \cup \{g,\sim g\}$ es posible derivar ~g y g.

Relaciones entre argumentos: desacuerdo

Dado un programa (Π, Δ) dos argumentos <X,f> y <Y,g> están en desacuerdo (o en conflicto) si $\{f,g\} \cup \Pi$ es un conjunto contradictorio (deriva dos literales complementarios).

A partir de (Π_2, Δ_2) se pueden construir < **A**,g> < **B**,f> <**C**,~g> y <**D**,e>

$\Pi_2 = \{ \sim p \leftarrow g.,\ p \leftarrow f.,\ b.,\ a. \}$

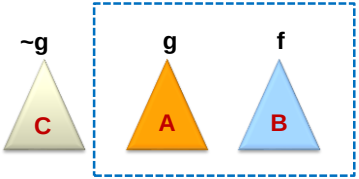
$\Delta_2 = \{ g -< a.,\ f -< b.,\ \sim g -< b.,\ e -< f. \}$

A = { g–< a. }

B = { f–< b. }

C = { ~g–< b. }

D = { e–< f, f–< b }



< **A**,g> y <**C**,~g> están en desacuerdo ya que a partir de $\Pi_2 \cup \{g,\sim g\}$ es posible derivar ~g y g.

< **A**,g> y <**B**,f> están en desacuerdo ya que a partir de $\Pi_2 \cup \{f, g\}$ es posible derivar ~p y p.

Relaciones entre argumentos: desacuerdo

Dado un programa (Π, Δ) dos argumentos <X,f> y <Y,g> están en desacuerdo (o en conflicto) si $\{f,g\} \cup \Pi$ es un conjunto contradictorio (deriva dos literales complementarios).

A partir de (Π_2, Δ_2) se pueden construir < **A**,g> < **B**,f> <**C**,~g> y <**D**,e>

$\Pi_2 = \{ \sim p \leftarrow g.,\ p \leftarrow f.,\ b.,\ a. \}$

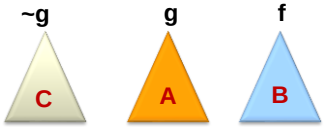
$\Delta_2 = \{ g -\!< a.,\ f -\!< b.,\ \sim g -\!< b.,\ e -\!< f. \}$

A = { g–< a. }

B = { f–< b. }

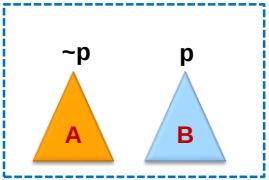
C = { ~g–< b. }

D = { e–< f, f–< b }



< **A**,g> y <**C**,~g> están en desacuerdo ya que a partir de $\Pi_2 \cup \{g,\sim g\}$ es posible derivar ~g y g.

< **A**,g> y <**B**,f> están en desacuerdo ya que a partir de $\Pi_2 \cup \{f, g\}$ es posible derivar ~p y p.



Observá que a partir de (Π_2, Δ_2) también se pueden construir < **A**,~p > y < **B**, p >

Relaciones entre argumentos:

sub-argumento

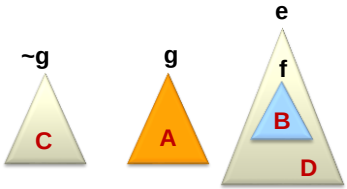
Relaciones entre argumentos: sub-argumento

Un argumento <**S**,q> es un sub-argumento de <**X**,h> si se cumple que **S** $\subseteq$ **X** .

A partir de (Π_2, Δ_2) se pueden construir < **A**,g> < **B**,f> <**C**,~g> y <**D**,e>

$$\Pi_2 = \left\{ \begin{array}{l} \sim p \leftarrow g. \\ p \leftarrow f. \\ b. \\ a. \end{array} \right\} \quad \Delta_2 = \left\{ \begin{array}{l} g \mathrel{|\!\sim} a. \\ f \mathrel{|\!\sim} b. \\ \sim g \mathrel{|\!\sim} b. \\ e \mathrel{|\!\sim} f. \end{array} \right\}$$

$$\mathbf{A} = \{ g \mathrel{|\!\sim} a. \} \quad \mathbf{B} = \{ f \mathrel{|\!\sim} b. \} \quad \mathbf{C} = \{ \sim g \mathrel{|\!\sim} b. \} \quad \mathbf{D} = \{ e \mathrel{|\!\sim} f,\ f \mathrel{|\!\sim} b \}$$



Notación gráfica < **B**, f > es un sub-argumento de < **D**,e >

Ejemplos:

< **B**, f > es un sub-argumento (propio) de < **D**, e >

< **A**, g > es un sub-argumento de < **A**, g >.

Observación: todo argumento es sub-argumento de sí mismo.

Relaciones entre argumentos:

contra-argumento

Ataque (contra-argumento)

El argumento <**X**,p> ataca (contra-argumenta) a <**Y**,h> si existe un sub-argumento <**S**,q> de <**Y**,h> (llamado sub-argumento de desacuerdo) tal que <**X**,p> y <**S**,q> están en desacuerdo.

El sub-argumento <**S**,q> puede ser el propio <**Y**,h>.

El literal q generalmente es llamado punto de ataque.

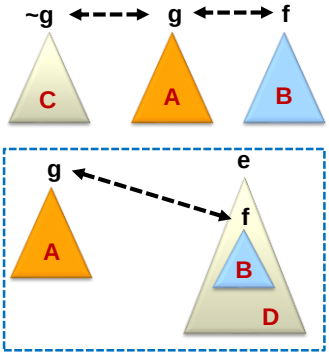
$\Pi_2 = \{ \sim p \leftarrow g.,\ p \leftarrow f.,\ b.,\ a. \}$ $\Delta_2 = \{ g \mathrel{-\!\!<} a.,\ f \mathrel{-\!\!<} b.,\ \sim g \mathrel{-\!\!<} b.,\ e \mathrel{-\!\!<} f. \}$

A = { $g \mathrel{-\!\!<} a.$ }

B = { $f \mathrel{-\!\!<} b.$ }

C = { $\sim g \mathrel{-\!\!<} b.$ }

D = { $e \mathrel{-\!\!<} f,\ f \mathrel{-\!\!<} b$ }



Ejemplos:

Los argumentos <**A**,g> y <**C**,~g> se atacan uno al otro.

Los argumentos <**A**,g> y <**B**, f> se atacan uno al otro ya que $\{ g,f \} \cup \Pi_2$ es contradictorio (deriva p y ~p).

El argumento <**A**,g> ataca a <**D**,e> con sub-argumento de desacuerdo <**B**,f>

Relaciones entre argumentos:

preferencia

Preferencia entre argumentos

Si se dispone de un orden parcial (>) entre algunas reglas rebatibles del programa, entonces se puede definir un criterio de preferencia entre argumentos llamado "prioridad entre reglas" de la siguiente manera:

Un argumento **N** es preferido a otro argumento **A** si se cumplen estas dos condiciones:

1. existe R_1 en **N** y R_2 en **A** tal que $R_1 > R_2$ (**N** tiene al menos una regla mejor que una de las de **A**)
2. no existe R_1 en **N** y R_2 en **A** tal que $R_1 < R_2$ (**A** no tiene ninguna regla mejor que **N**)

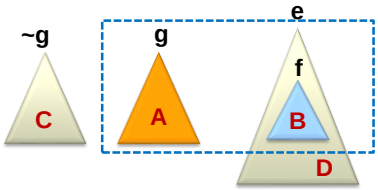
$\Pi_2 = \{ \sim p \leftarrow g.,\ p \leftarrow f.,\ b.,\ a. \}$ $\Delta_2 = \{ g \prec a.,\ f \prec b.,\ \sim g \prec b.,\ e \prec f. \}$

A = { $g \prec a.$ }

B = { $f \prec b.$ }

C = { $\sim g \prec b.$ }

D = { $e \prec f$, $f \prec b$ }



Orden $>_3$ entre reglas de Δ_2
$(g \prec a) >_3 (f \prec b)$

Con $>_3$, el argumento A es preferido al argumento B ya que se cumple la condición 1: hay una regla en A que es mejor que una de B, y también se cumple la condición 2 (no hay ninguna regla en B que sea mejor que una de A).

Relaciones entre argumentos:

derrota propia y por bloqueo

Derrota propia y derrota por bloqueo

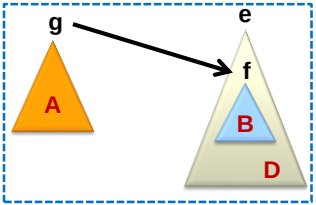
El argumento <**X**,p> derrota a <**Y**,h> si <**X**,p> ataca a <**Y**,h> con sub-argumento de desacuerdo <**S**,q> y se cumple una de estas dos condiciones:

(a) <**X**,p> es preferido a <**S**,q> (es un derrotador propio),

(b) <**X**,p> no es comparable con <**S**,q> (es un derrotador por bloqueo)

La relación de derrota combina ataque y preferencia.

Observá que la preferencia es entre el que ataca <**X**,p> y el sub-argumento <**S**,q>.



$$\Pi_2 = \left\{ \begin{array}{l} \sim p \leftarrow g. \\ p \leftarrow f. \\ b. \\ a. \end{array} \right\} \quad \Delta_2 = \left\{ \begin{array}{l} g -\!< a. \\ f -\!< b. \\ \sim g -\!< b. \\ e -\!< f. \end{array} \right\}$$

$$\mathbf{A} = \{\, g -\!< a. \,\} \quad \mathbf{C} = \{\, \sim g -\!< b. \,\}$$

$$\mathbf{B} = \{\, f -\!< b. \,\} \quad \mathbf{D} = \{\, f -\!< b ,\ e -\!< f \,\}$$

Orden $>_3$ entre reglas de Δ_2
$(g -\!< a) >_3 (f -\!< b.)$

A es preferido a **B**



<**A**,g> es un derrotador propio de <**D**,e> ya que <**A**,g> ataca en el sub-argumento de desacuerdo <**B**,f> y usando prioridad entre reglas <**A**,g> es preferido a <**B**,f>.

Tambien se cumple que <**A**,g> es derrotador propio de <**B**,f>.

Derrota propia y derrota por bloqueo

El argumento <X,p> derrota a <Y,h> si <X,p> ataca a <Y,h> con sub-argumento de desacuerdo <S,q> y se cumple una de estas dos condiciones:

(a) <X,p> es preferido a <S,q> (es un derrotador propio),

(b) <X,p> no es comparable con <S,q> (es un derrotador por bloqueo)

La relación de derrota combina ataque y preferencia.

Observá que la preferencia es entre el que ataca <X,p> y el sub-argumento <S,q>.

$\Pi_2 = \{ \sim p \leftarrow g.,\ p \leftarrow f.,\ b.,\ a. \}$ $\Delta_2 = \{ g -\!< a.,\ f -\!< b.,\ \sim g -\!< b.,\ e -\!< f. \}$

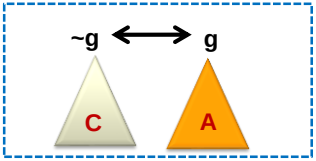
A = { g –< a. } **C** = { ~g –< b. }

B = { f –< b. } **D** = { f –< b , e –< f }

Orden $>_3$ entre reglas de Δ_2

$(g -\!< a) >_3 (f -\!< b.)$

A y **C** son incomparables





<C,~g> es derrotador por bloqueo de <A,g> (y viceversa) ya que se atacan mutuamente y no son comparables

Derrota propia y derrota por bloqueo

El argumento <**X**,p> derrota a <**Y**,h> si <**X**,p> ataca a <**Y**,h> con sub-argumento de desacuerdo <**S**,q> y se cumple una de estas dos condiciones:

(a) <**X**,p> es preferido a <**S**,q> (es un derrotador propio),

(b) <**X**,p> no es comparable con <**S**,q> (es un derrotador por bloqueo)

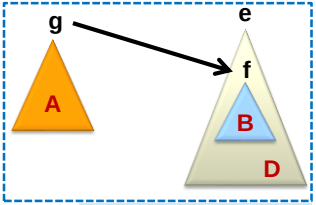
La relación de derrota combina ataque y preferencia.

Observá que la preferencia es entre el que ataca <**X**,p> y el sub-argumento <**S**,q>.

$\Pi_2 = \{ \sim p \leftarrow g.,\ p \leftarrow f.,\ b.,\ a. \}$ $\Delta_2 = \{ g -\!\prec a.,\ f -\!\prec b.,\ \sim g -\!\prec b.,\ e -\!\prec f. \}$

A = { $g -\!\prec a.$ } **C** = { $\sim g -\!\prec b.$ }

B = { $f -\!\prec b.$ } **D** = { $f -\!\prec b$, $e -\!\prec f$ }



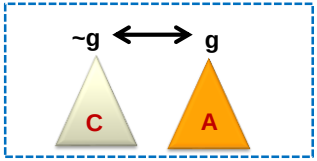
derrotador propio

Orden $>_3$ entre reglas de Δ_2

$(g -\!\prec a) >_3 (f -\!\prec b.)$

A es preferido a **B**

A y **C** son incomparables



derrota por bloqueo

B ataca a **A**, pero **B** **no derrota** a **A** porque **A** es preferido a **B**.



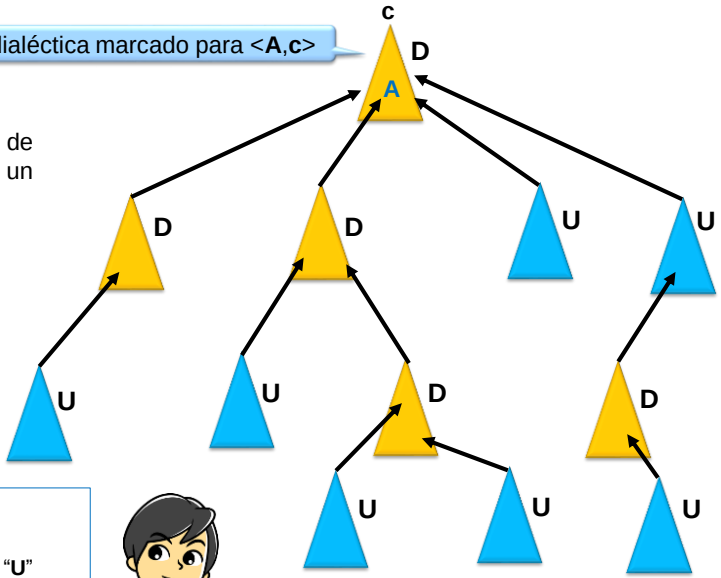
Tercera Parte:

Inferencia Argumentativa

Árbol de dialéctica marcado



Para determinar si la raíz de un árbol de dialéctica resulta derrotada o no, DeLP realiza un proceso de marcado de los nodos.



Marcas posibles para un nodo:

D: **Defeated** (derrotado)

U: **Undefeated** (no derrotado)

Proceso de marcado:

Toda hoja es marcada "**U**" y todo nodo interno:

- es marcado "**D**" si tiene al menos un hijo marcado "**U**"
- es marcado "**U**" si todos sus hijos están marcados "**D**"

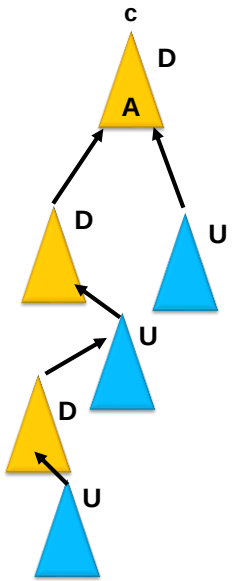


Conclusión garantizada

Una conclusión **c** estará garantizada por un programa DeLP, si existe al menos un árbol de dialéctica para un argumento que sustenta **c**, cuya raíz está marcada como **U**.

Ante una consulta **c,** DeLP analizará los argumentos que existan para **c** y los que existan para **~c**.

Veamos paso a paso como es el proceso de determinar una garantía, explorando los árboles de dialéctica en profundidad.



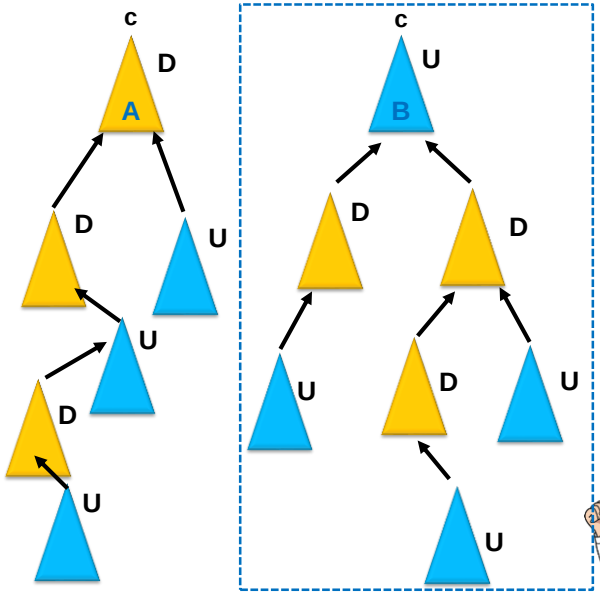




Conclusión garantizada

Una conclusión **c** estará garantizada por un programa DeLP, si existe al menos un árbol de dialéctica para un argumento que sustenta **c**, cuya raíz está marcada como **U**.

Ante una consulta **c,** DeLP analizará los argumentos que existan para **c** y los que existan para **~c**.







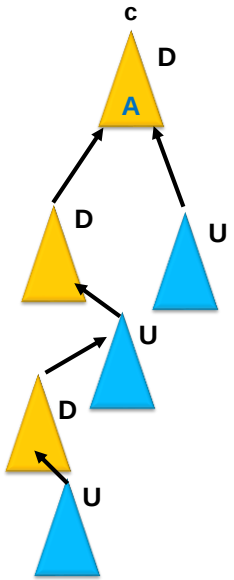
Conclusión garantizada

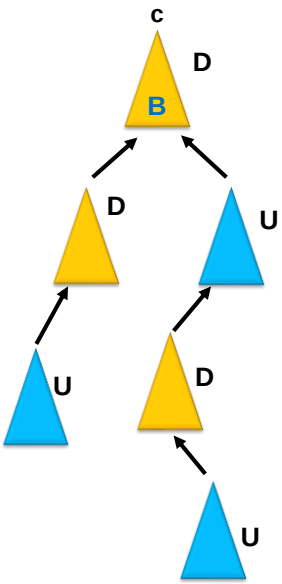
Una conclusión **c** estará garantizada por un programa DeLP, si existe al menos un árbol de dialéctica para un argumento que sustenta **c**, cuya raíz está marcada como **U**.

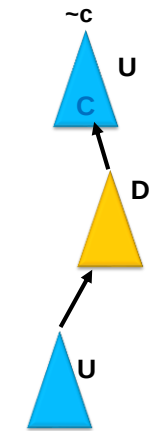
Ante una consulta **c,** DeLP analizará los argumentos que existan para **c** y los que existan para **~c**.



En este ejemplo, hay una garantía para el literal **~c**.









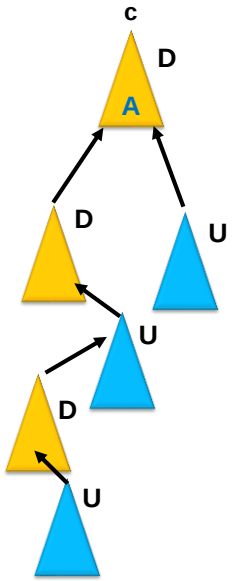
Situación de indecisión

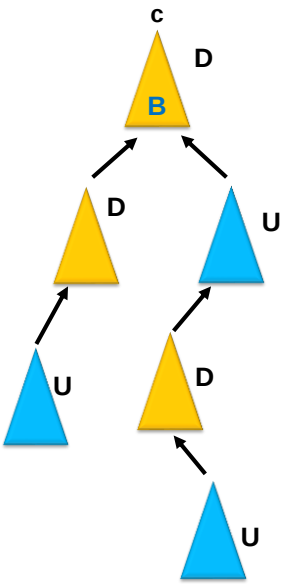
Una conclusión **c** estará garantizada por un programa DeLP, si existe al menos un árbol de dialéctica para un argumento que sustenta **c**, cuya raíz está marcada como **U**.

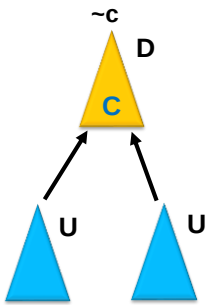
Ante una consulta **c,** DeLP analizará los argumentos que existan para **c** y los que existan para **~c**.

En este ejemplo, no hay una garantía para el literal **c**, y tampoco hay una garantía para el literal **~c**.











Respuestas a una consulta

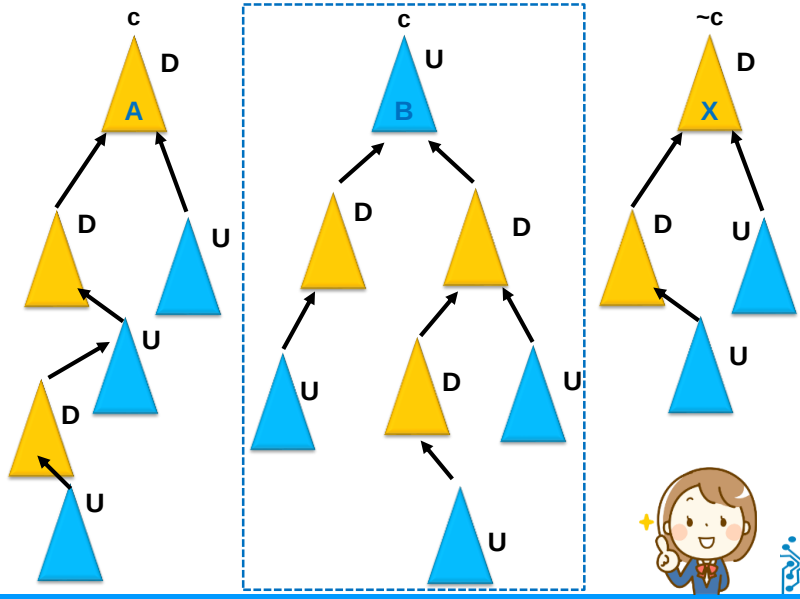
Respuesta a una consulta

La respuesta de un intérprete para una consulta **c** a partir de un PLR **P** es:

yes: cuando **c** está garantizada por **P**.

Los árboles de dialéctica dan, además, una explicación a las respuestas.

Una conclusión **c** estará garantizada por un programa DeLP, si existe al menos un árbol de dialéctica para un argumento que sustenta **c**, cuya raíz está marcada como **U**.



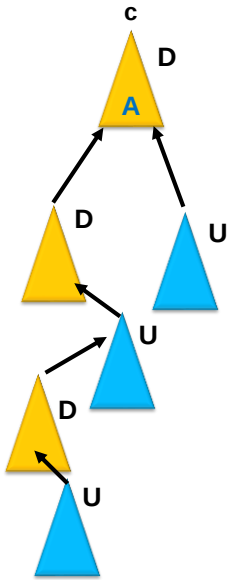
Respuesta a una consulta

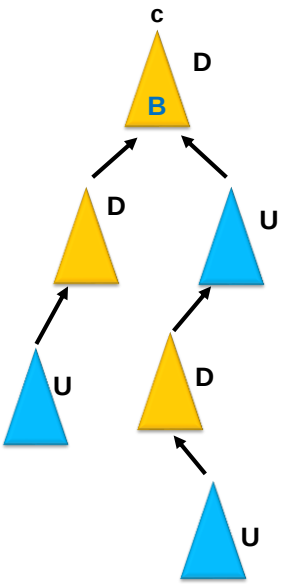
La respuesta de un intérprete para una consulta **c** a partir de un PLR **P** es:

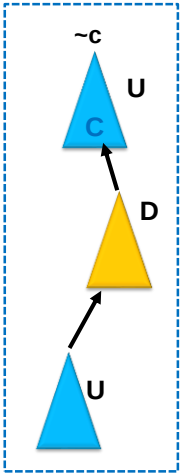
yes: cuando **c** está garantizada por **P**.

no: cuando el *complemento*[1] de **c** está garantizado por **P**.

Los árboles de dialéctica dan, además, una explicación a las respuestas.









[1] El complemento de un literal con respecto a "~" (negación fuerte) se define como: *complemento(c) = ~c* y *complemento(~c) = c*.

Respuesta a una consulta

La respuesta de un intérprete para una consulta **c** a partir de un PLR **P** es:

yes: cuando **c** está garantizada por **P**.

no: cuando el *complemento*[1] de **c** está garantizado por **P**.

unknown: cuando **c** no *pertenece*[2] a **P**.

[1] El complemento de un literal con respecto a "~" (negación fuerte) se define como: *complemento(c) = ~c* y *complemento(~c) = c*.

[2] Un literal *c* pertenece a un programa P cuando *c o complemento(c)* está presente en alguna regla de P.



Respuesta a una consulta

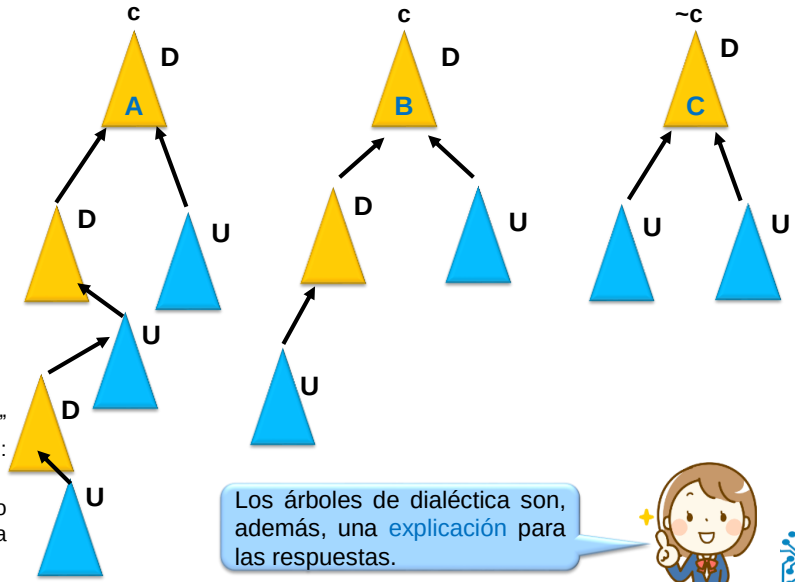
La respuesta de un intérprete para una consulta **c** a partir de un PLR **P** es:

yes: cuando **c** está garantizada por **P**.

no: cuando el *complemento*[1] de **c** está garantizado por **P**.

unknown: cuando **c** no *pertenece*[2] a **P**.

undecided: cuando **c** pertenece a **P**, pero **c** no está garantizada por **P** y su complemento tampoco está garantizado por **P**.



Los árboles de dialéctica son, además, una explicación para las respuestas.

[1] El complemento de un literal con respecto a "~" (negación fuerte) se define como: *complemento(c) = ~c* y *complemento(~c) = c*.

[2] Un literal *c* pertenece a un programa P cuando *c o complemento(c)* está presente en alguna regla de P.

Respuestas en el PLR Merval



¿Qué repuesta tiene comprar_acciones(acme)?

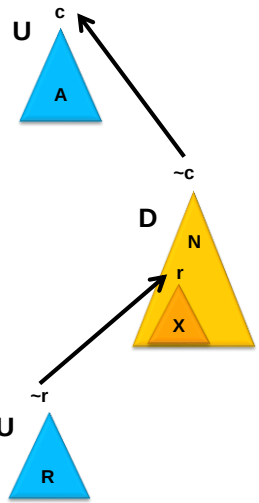
$R2 >_m R1$
$R7 >_m R5$

Δ_m

R1 comprar_acciones(X) –< buen_precio(X).
R2 ~comprar_acciones(X) –< riesgosa (X).
R3 buen_precio(X) –< buen_precio_web(X).
~buen_precio(X) –< en_baja_web(X).
R5 riesgosa(X) –< en_fusion(X,Y).
R7 riesgosa(X) –< rumores_deuda_web(X).
~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m

~riesgosa(X) ← fuerte(X).
fuerte(X) ← listado_fuertes_web(X).
con_deuda(X) ← balance_negativo(X).
en_fusion(X,Y) ← fusiones_web(X,Y).
en_baja_web(skynet).
buen_precio_web(skynet).
buen_precio_web(acme).
fusiones_web(acme, strong).
listado_fuertes_web(strong).



La respuesta para “comprar_acciones(acme)” es “**sí**”



<A, comprar_acciones(acme) >

A = { comprar_acciones(acme) –< buen_precio(acme). R1
buen_precio(acme) –< buen_precio_web(acme). R3 }

Como **R2 >$_m$ R1,** entonces **N** es preferido a **A**, y por lo tanto, **N** es una derrota propia para **A**.

<N, ~comprar_acciones(acme) >

N = { ~comprar_acciones(acme) –< riesgosa(acme). R2
riesgosa(acme) –< en_fusion(acme,strong). R5 }

< X, riesgosa(acme) >

X = { riesgosa(acme) –< en_fusion(acme,strong). R5 }

Como **R7 >$_m$ R5,** entonces **R** es preferido a **X**, y por lo tanto, **R** es una derrota propia para **N**.

< R, ~riesgosa(acme) >

R7

R = { ~riesgosa(acme) –<en_fusion(acme,strong), fuerte(strong) }

Respuestas en el PLR Merval

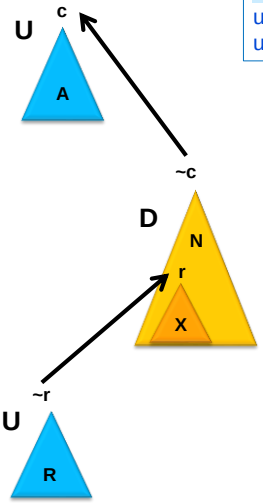


¿Qué repuesta tiene ~comprar_acciones(acme)?

$R2 >_m R1$
$R7 >_m R5$

R1 comprar_acciones(X) –< buen_precio(X).
R2 ~comprar_acciones(X) –< riesgosa (X).
R3 buen_precio(X) –< buen_precio_web(X).
Δ_m
~buen_precio(X) –< en_baja_web(X).
R5 riesgosa(X) –< en_fusion(X,Y).
riesgosa(X) –< rumores_deuda_web(X).
R7 ~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m
~riesgosa(X) ← fuerte(X).
fuerte(X) ← listado_fuertes_web(X).
con_deuda(X) ← balance_negativo(X).
en_fusion(X,Y) ← fusiones_web(X,Y).
en_baja_web(skynet).
buen_precio_web(skynet).
buen_precio_web(acme).
fusiones_web(acme, strong).
listado_fuertes_web(strong).



Respuesta a una consulta h en un programa P.
yes: cuando h está garantizado por P,
no: cuando el complemento de h está garantizado por P,
unknown: cuando el literal h no pertenece a P,
undecided: si no se cumple ninguna de las anteriores.

La respuesta es “**no**”



Consulta	Respuesta
comprar_acciones(acme)	yes
~comprar_acciones(acme)	no

Respuestas en el PLR Merval

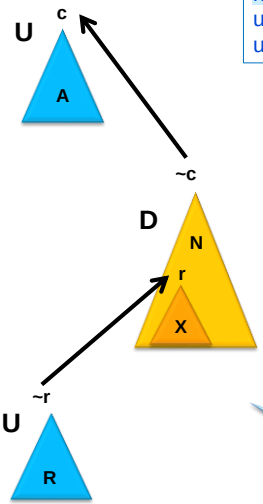
¿Qué repuesta tiene ~comprar_acciones(acme)?



R2 >$_m$ R1
R7 >$_m$ R5

R1 comprar_acciones(X) –< buen_precio(X).
R2 ~comprar_acciones(X) –< riesgosa (X).
R3 buen_precio(X) –< buen_precio_web(X).
Δ_m
~buen_precio(X) –< en_baja_web(X).
R5 riesgosa(X) –< en_fusion(X,Y).
R7 riesgosa(X) –< rumores_deuda_web(X).
~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m
~riesgosa(X) ← fuerte(X).
fuerte(X) ← listado_fuertes_web(X).
con_deuda(X) ← balance_negativo(X).
en_fusion(X,Y) ← fusiones_web(X,Y).
en_baja_web(skynet).
buen_precio_web(skynet).
buen_precio_web(acme).
fusiones_web(acme, strong).
listado_fuertes_web(strong).



Respuesta a una consulta h en un programa P.
yes: cuando h está garantizado por P,
no: cuando el complemento de h está garantizado por P,
unknown: cuando el literal h no pertenece a P,
undecided: si no se cumple ninguna de las anteriores.

La respuesta es “**no**”



Consulta	Respuesta
comprar_acciones(acme)	yes
~comprar_acciones(acme)	no

Observá que con el mismo árbol de dialéctica estamos explicando las dos respuestas.

Respuestas en el PLR Merval



¿Qué repuesta tiene vender_acciones(acme)?

$R2 >_m R1$
$R7 >_m R5$

R1 comprar_acciones(X) –< buen_precio(X).
R2 ~comprar_acciones(X) –< riesgosa (X).
R3 buen_precio(X) –< buen_precio_web(X).
Δ_m
R5 riesgosa(X) –< en_fusion(X,Y).
R7 ~buen_precio(X) –< en_baja_web(X).
riesgosa(X) –< rumores_deuda_web(X).
~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m
~riesgosa(X) ← fuerte(X).
fuerte(X) ← listado_fuertes_web(X).
con_deuda(X) ← balance_negativo(X).
en_fusion(X,Y) ← fusiones_web(X,Y).
en_baja_web(skynet).
buen_precio_web(skynet).
buen_precio_web(acme).
fusiones_web(acme, strong).
listado_fuertes_web(strong).

Respuesta a una consulta h en un programa P.
yes: cuando h está garantizado por P,
no: cuando el complemento de h está garantizado por P,
unknown: cuando el literal h no pertenece a P,
undecided: si no se cumple ninguna de las anteriores.

La respuesta es “**unknown**”



Consulta	Respuesta
comprar_acciones(acme)	yes
~comprar_acciones(acme)	no
vender_acciones(acme)	unknown

Respuestas en el PLR Merval

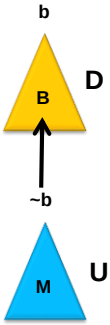


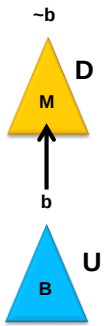
¿Qué repuesta tiene buen_precio(skynet)?

$R2 >_m R1$
$R7 >_m R5$

comprar_acciones(X) –< buen_precio(X).
~comprar_acciones(X) –< riesgosa (X).
R3 buen_precio(X) –< buen_precio_web(X).
R4 ~buen_precio(X) –< en_baja_web(X).
riesgosa(X) –< en_fusion(X,Y).
riesgosa(X) –< rumores_deuda_web(X).
~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m
~riesgosa(X) ← fuerte(X).
fuerte(X) ← listado_fuertes_web(X).
con_deuda(X) ← balance_negativo(X).
en_fusion(X,Y) ← fusiones_web(X,Y).
en_baja_web(skynet).
buen_precio_web(skynet).
buen_precio_web(acme).
fusiones_web(acme, strong).
listado_fuertes_web(strong).





Respuesta a una consulta h en un programa P.
yes: cuando h está garantizado por P,
no: cuando el complemento de h está garantizado por P,
unknown: cuando el literal h no pertenece a P,
undecided: si no se cumple ninguna de las anteriores.



Consulta	Respuesta
comprar_acciones(acme)	yes
~comprar_acciones(acme)	no
vender_acciones(acme)	unknown
buen_precio (skynet)	
~buen_precio (skynet)	

Respuestas en el PLR Merval



¿Qué repuesta tiene buen_precio(skynet)?

$R2 >_m R1$
$R7 >_m R5$

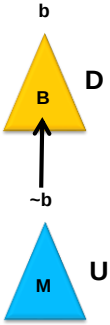
R3

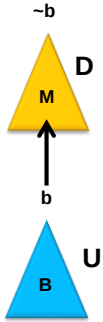
R4

comprar_acciones(X) –< buen_precio(X).
~comprar_acciones(X) –< riesgosa (X).
buen_precio(X) –< buen_precio_web(X).
~buen_precio(X) –< en_baja_web(X).
riesgosa(X) –< en_fusion(X,Y).
riesgosa(X) –< rumores_deuda_web(X).
~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m

~riesgosa(X) ← fuerte(X).
fuerte(X) ← listado_fuertes_web(X).
con_deuda(X) ← balance_negativo(X).
en_fusion(X,Y) ← fusiones_web(X,Y).
en_baja_web(skynet).
buen_precio_web(skynet).
buen_precio_web(acme).
fusiones_web(acme, strong).
listado_fuertes_web(strong).





Respuesta a una consulta h en un programa P.
yes: cuando h está garantizado por P,
no: cuando el complemento de h está garantizado por P,
unknown: cuando el literal h no pertenece a P,
undecided: si no se cumple ninguna de las anteriores.



La respuesta es “**undecide**”

Consulta	Respuesta
comprar_acciones(acme)	yes
~comprar_acciones(acme)	no
vender_acciones(acme)	unknown
buen_precio (skynet)	undecided
~buen_precio (skynet)	undecided

Comentario importante sobre las marcas de un argumento

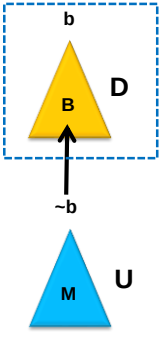


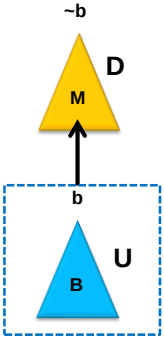
¿Qué repuesta tiene buen_precio(skynet)?

$R2 >_m R1$
$R7 >_m R5$

comprar_acciones(X) –< buen_precio(X).
~comprar_acciones(X) –< riesgosa (X).
R3 buen_precio(X) –< buen_precio_web(X).
R4 ~buen_precio(X) –< en_baja_web(X).
riesgosa(X) –< en_fusion(X,Y).
riesgosa(X) –< rumores_deuda_web(X).
~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m
~riesgosa(X) ← fuerte(X).
fuerte(X) ← listado_fuertes_web(X).
con_deuda(X) ← balance_negativo(X).
en_fusion(X,Y) ← fusiones_web(X,Y).
en_baja_web(skynet).
buen_precio_web(skynet).
buen_precio_web(acme).
fusiones_web(acme, strong).
listado_fuertes_web(strong).





Respuesta a una consulta h en un programa P.
yes: cuando h está garantizado por P,
no: cuando el complemento de h está garantizado por P,
unknown: cuando el literal h no pertenece a P,
undecided: si no se cumple ninguna de las anteriores.

Observá que el mismo argumento puede estar con diferente marca si en árboles diferentes.



Comentario importante sobre las marcas de un argumento

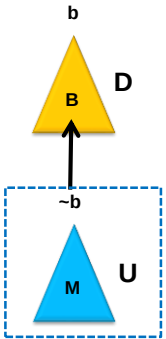


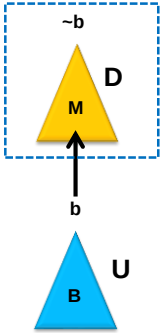
¿Qué repuesta tiene buen_precio(skynet)?

R2 >$_m$ R1
R7 >$_m$ R5

comprar_acciones(X) –< buen_precio(X).
~comprar_acciones(X) –< riesgosa (X).
R3 buen_precio(X) –< buen_precio_web(X).
R4 ~buen_precio(X) –< en_baja_web(X).
riesgosa(X) –< en_fusion(X,Y).
riesgosa(X) –< rumores_deuda_web(X).
~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m
~riesgosa(X) ← fuerte(X).
fuerte(X) ← listado_fuertes_web(X).
con_deuda(X) ← balance_negativo(X).
en_fusion(X,Y) ← fusiones_web(X,Y).
en_baja_web(skynet).
buen_precio_web(skynet).
buen_precio_web(acme).
fusiones_web(acme, strong).
listado_fuertes_web(strong).





Respuesta a una consulta h en un programa P.
yes: cuando h está garantizado por P,
no: cuando el complemento de h está garantizado por P,
unknown: cuando el literal h no pertenece a P,
undecided: si no se cumple ninguna de las anteriores.

Observá que el mismo argumento puede estar con diferente marca si en árboles diferentes.



¿ Argumentación infinita ?

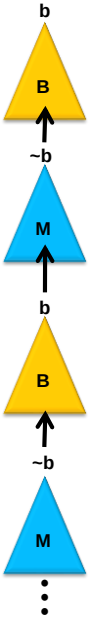


¿Qué repuesta tiene buen_precio(skynet)?

$R2 >_m R1$
$R7 >_m R5$

comprar_acciones(X) –< buen_precio(X).
~comprar_acciones(X) –< riesgosa (X).
R3 buen_precio(X) –< buen_precio_web(X).
R4 ~buen_precio(X) –< en_baja_web(X).
riesgosa(X) –< en_fusion(X,Y).
riesgosa(X) –< rumores_deuda_web(X).
~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m
~riesgosa(X) ← fuerte(X).
fuerte(X) ← listado_fuertes_web(X).
con_deuda(X) ← balance_negativo(X).
en_fusion(X,Y) ← fusiones_web(X,Y).
en_baja_web(skynet).
buen_precio_web(skynet).
buen_precio_web(acme).
fusiones_web(acme, strong).
listado_fuertes_web(strong).



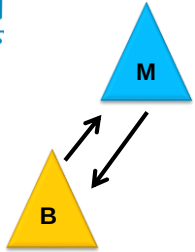
Respuesta a una consulta h en un programa P.
yes: cuando h está garantizado por P,
no: cuando el complemento de h está garantizado por P,
unknown: cuando el literal h no pertenece a P,
undecided: si no se cumple ninguna de las anteriores.

¿Qué evita que B vuelva a derrotar a M o M a vuelva a derrotar a B?

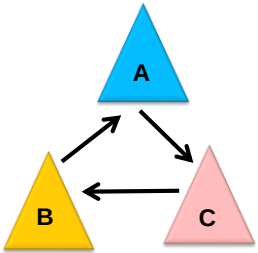


Líneas de argumentación aceptables

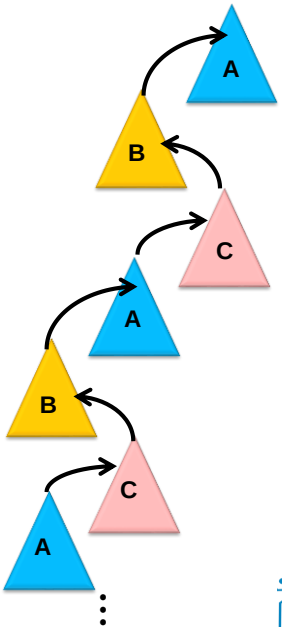
Situaciones que debemos evitar…







Hay que considerar como resolver situaciones “circulares”





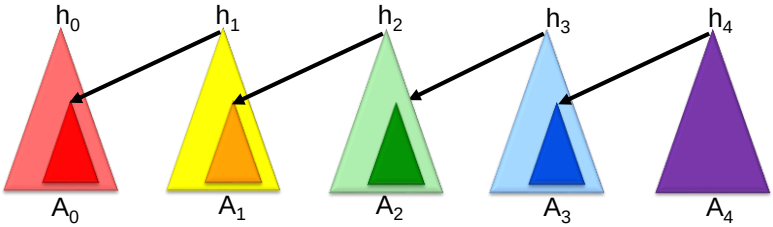
Hay que evitar una línea de argumentación infinita.

Línea de argumentación para un argumento

Una línea de argumentación para un argumento <A_0,h_0>, es una secuencia finita de argumentos que comienza con A_0 y a partir del segundo, cada argumento es un derrotador del anterior en la secuencia.

En una línea de argumentación, el segundo argumento derrota al primero, pero como el tercero derrota al segundo entonces defiende al primero.

En toda línea de argumentación podemos identificar argumentos de interferencia para el primero de la línea (los que lo derrotan directa o indirectamente) y argumentos de soporte para el primero (los que lo defienden).



Argumentos de soporte para <A_0,h_0> = {A_0, A_2, A_4}

Argumentos de interferencia para <A_0,h_0> = {A_1, A_3}

Líneas de argumentación en un árbol de dialéctica

En un árbol de dialéctica para un argumento A_0 a partir de un programa P, todo camino de la raíz a una hoja es una línea de argumentación.

Líneas de argumentación del árbol de dialéctica para el argumento **<A_0,c>**

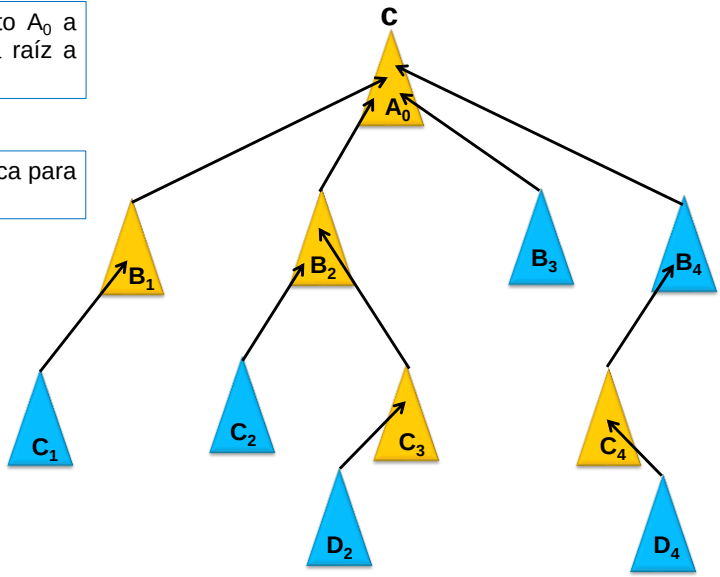
L_1=[A_0, B_1, C_1]

L_2=[A_0, B_2, C_2]

L_3=[A_0, B_2, C_3, D_2]

L_4=[A_0, B_3]

L_5=[A_0, B_4, C_4, D_4]



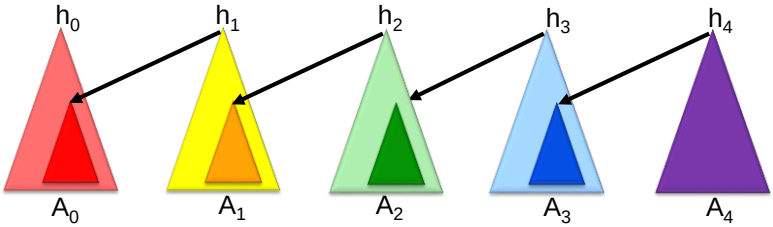
Línea de argumentación aceptable

Una línea de argumentación para $\mathbf{A_0}$ es aceptable si:

(1) Ningún argumento $\mathbf{A_k}$ aparece más de una vez en la línea, o es un sub-argumento de desacuerdo de otro $\mathbf{A_i}$ que aparece previamente (i<k).

(2) El conjunto de argumentos de soporte para $\mathbf{A_0}$ no es contradictorio (ídem el de interferencia)

(3) Si $\mathbf{A_k}$ es un derrotador de bloqueo para $\mathbf{A_{k-1}}$, entonces $\mathbf{A_{k+1}}$ debe ser derrotador propio para $\mathbf{A_k}$ (no debe haber dos derrotadores de bloqueo consecutivos).



En **DeLP,** en un árbol de dialéctica, toda línea de argumentación debe cumplir tres condiciones que la hacen “aceptable” e impiden incurrir en situaciones no deseadas.

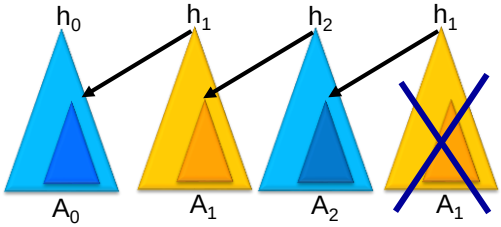
(1) evitando líneas con ciclos (infinitos)

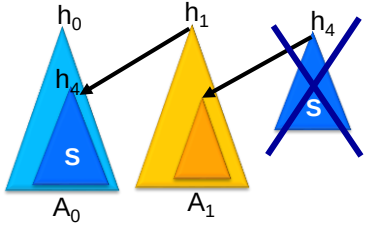
Una línea de argumentación para $\mathbf{A_0}$ es aceptable si:

(1) Ningún argumento $\mathbf{A_k}$ aparece más de una vez en la línea, o es un sub-argumento de desacuerdo de otro $\mathbf{A_i}$ que aparece previamente (i<k).

(2) El conjunto de argumentos de soporte para $\mathbf{A_0}$ no es contradictorio (ídem el de interferencia)

(3) Si $\mathbf{A_k}$ es un derrotador de bloqueo para $\mathbf{A_{k-1}}$, entonces $\mathbf{A_{k+1}}$ debe ser derrotador propio para $\mathbf{A_k}$ (no debe haber dos derrotadores de bloqueo consecutivos).





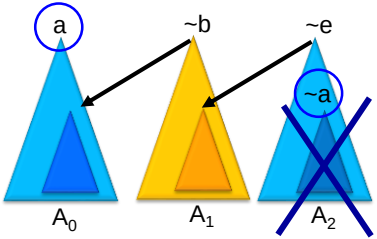
(2) evitando contradicciones

Una línea de argumentación para $\mathbf{A_0}$ es aceptable si:

(1) Ningún argumento $\mathbf{A_k}$ aparece más de una vez en la línea, o es un sub-argumento de desacuerdo de otro $\mathbf{A_i}$ que aparece previamente (i<k).

(2) El conjunto de argumentos de soporte para $\mathbf{A_0}$ no es contradictorio [1] (ídem el de interferencia)

(3) Si $\mathbf{A_k}$ es un derrotador de bloqueo para $\mathbf{A_{k-1}}$, entonces $\mathbf{A_{k+1}}$ debe ser derrotador propio para $\mathbf{A_k}$ (no debe haber dos derrotadores de bloqueo consecutivos).



El argumento A_2 no debe agregarse a la línea porque es contradictorio con A_0

$A_0 = \{ a -< b.\quad b -< c. \}$

$A_1 = \{ \sim b -< e.\quad e -< d. \}$

$A_2 = \{ \sim e -< \sim a.\quad \sim a -< j. \}$

$(\sim e -< \sim a) > (e -< d)$

$\Delta = \{ a -< b.,\ b -< c.,\ \sim b -< e.,\ e -< d.,\ \sim e -< \sim a.,\ \sim a -< j. \}$

$\Pi = \{ c.,\ d.,\ j. \}$

[1] Un conjunto de argumentos $\{A_1 \ldots A_N\}$ de un programa (Π, Δ) es contradictorio, si la unión $\Pi \cup A_1 \cup \ldots \cup A_N$ permite derivar un par de literales complementarios p y ~p.

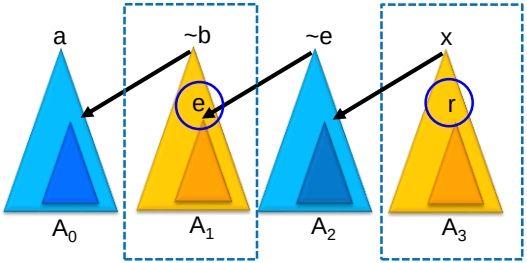
(2) evitando contradicciones

Una línea de argumentación para $\mathbf{A_0}$ es aceptable si:

(1) Ningún argumento $\mathbf{A_k}$ aparece más de una vez en la línea, o es un sub-argumento de desacuerdo de otro $\mathbf{A_i}$ que aparece previamente (i<k).

(2) El conjunto de argumentos de soporte para $\mathbf{A_0}$ no es contradictorio [1] (ídem el de interferencia)

(3) Si $\mathbf{A_k}$ es un derrotador de bloqueo para $\mathbf{A_{k-1}}$, entonces $\mathbf{A_{k+1}}$ debe ser derrotador propio para $\mathbf{A_k}$ (no debe haber dos derrotadores de bloqueo consecutivos).



$A_0 = \{ a\text{–}< b. \quad b\text{–}< c. \}$

$A_1 = \{ \sim b\text{–}< e. \quad e\text{–}< d. \}$

$A_2 = \{ \sim e\text{–}< \sim x. \ \sim x\text{–}< c. \}$

$A_3 = \{ x\text{–}< r. \quad r\text{–}< g. \}$

(~e–< ~x) > (e–< d)

$\Delta = \{$ a–< b. b–< c. ~b–< e. e–< d. ~e–< ~x. ~x–< c. x–< r. r–< g. $\}$

$\Pi = \{$ p ← r. ~p ← e. c. d. g. $\}$

[1] Un conjunto de argumentos $\{A_1 \ldots A_N\}$ de un programa (Π, Δ) es contradictorio, si la unión $\Pi \cup A_1 \cup \ldots \cup A_N$ permite derivar un par de literales complementarios p y ~p.

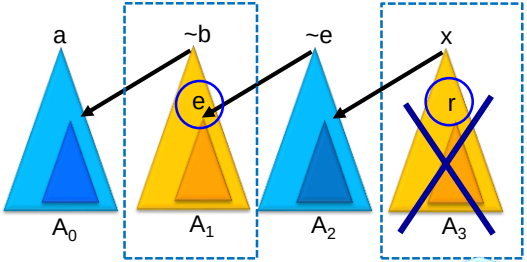
(2) evitando contradicciones

Una línea de argumentación para $\mathbf{A_0}$ es aceptable si:

(1) Ningún argumento $\mathbf{A_k}$ aparece más de una vez en la línea, o es un sub-argumento de desacuerdo de otro $\mathbf{A_i}$ que aparece previamente (i<k).

(2) El conjunto de argumentos de soporte para $\mathbf{A_0}$ no es contradictorio [1] (ídem el de interferencia)

(3) Si $\mathbf{A_k}$ es un derrotador de bloqueo para $\mathbf{A_{k-1}}$, entonces $\mathbf{A_{k+1}}$ debe ser derrotador propio para $\mathbf{A_k}$ (no debe haber dos derrotadores de bloqueo consecutivos).



$A_0 = \{ a \text{–<} b. \quad b \text{–<} c. \}$

$A_1 = \{ \sim b \text{–<} e. \quad e \text{–<} d. \}$

$A_2 = \{ \sim e \text{–<} \sim x. \ \sim x \text{–<} c. \}$

$A_3 = \{ x \text{–<} r. \quad r \text{–<} g. \}$

$(\sim e \text{–<} \sim x) > (e \text{–<} d)$

$\Delta = \{ a \text{–<} b.\ b \text{–<} c.\ \sim b \text{–<} e.\ e \text{–<} d.\ \sim e \text{–<} \sim x.\ \sim x \text{–<} c.\ x \text{–<} r.\ r \text{–<} g. \}$

$\Pi = \{ p \leftarrow r.\ \sim p \leftarrow e.\ c.\ d.\ g. \}$

Observá que $\Pi \cup A_1 \cup A_3$ es un conjunto contradictorio porque permite derivar p y ~p.

[1] Un conjunto de argumentos $\{A_1 \ldots A_N\}$ de un programa (Π, Δ) es contradictorio, si la unión $\Pi \cup A_1 \cup \ldots \cup A_N$ permite derivar un par de literales complementarios p y ~p.

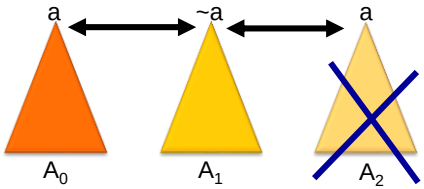
(3) evitando doble bloqueo

Una línea de argumentación para $\mathbf{A_0}$ es aceptable si:

(1) Ningún argumento $\mathbf{A_k}$ aparece más de una vez en la línea, o es un sub-argumento de desacuerdo de otro $\mathbf{A_i}$ que aparece previamente (i<k).

(2) El conjunto de argumentos de soporte para $\mathbf{A_0}$ no es contradictorio (ídem el de interferencia)

(3) Si $\mathbf{A_k}$ es un derrotador de bloqueo para $\mathbf{A_{k-1}}$, entonces $\mathbf{A_{k+1}}$ debe ser derrotador propio para $\mathbf{A_k}$ (no debe haber dos derrotadores de bloqueo consecutivos).



El argumento A_2 no debe agregarse a la línea por la condición 3.

$A_0 = \{ a\text{–}< b. \}$

$A_1 = \{ \sim a\text{–}< c. \}$

$A_2 = \{ a\text{–}< d. \}$

$(\sim e\text{–}< c) > (e\text{–}< d)$

$\Delta = \{ a\text{–}< b.,\ \sim a\text{–}< c.,\ a\text{–}< d.,\ e\text{–}< d.,\ \sim e\text{–}< c. \}$

$\Pi = \{ b., c., d. \}$

Como el bloqueo es simétrico, cuando $\mathbf{A_1}$ bloquea a $\mathbf{A_0}$ este último también bloquea a $\mathbf{A_1}$ por lo tanto $\mathbf{A_1}$ ya está bloqueado agregue o no al argumento $\mathbf{A_2}$

Árbol de dialéctica

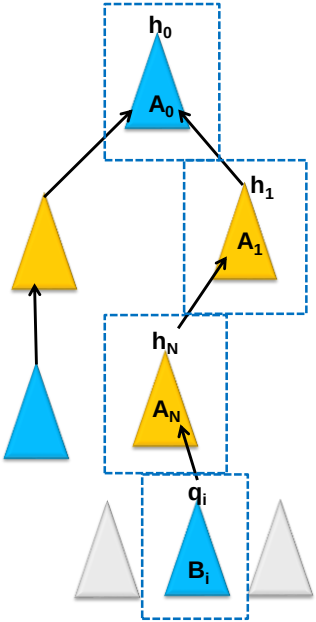
Definición de Árbol de dialéctica

Árbol de dialéctica para $<A_0, h_0>$:

1. La raíz del árbol es etiquetada con $<A_0, h_0>$
2. Dado un nodo cualquiera del árbol etiquetado $<A_N, h_N>$, donde $[<A_0,h_0>, <A_1,h_1>, ... <A_N, h_N>]$ es el camino desde la raíz hasta $<A_N, h_N>$.

   Sea $\{<B_1,q_1>,..., <B_k, q_k>\}$ el conjunto de todos los derrotadores de $<A_N, h_N>$.

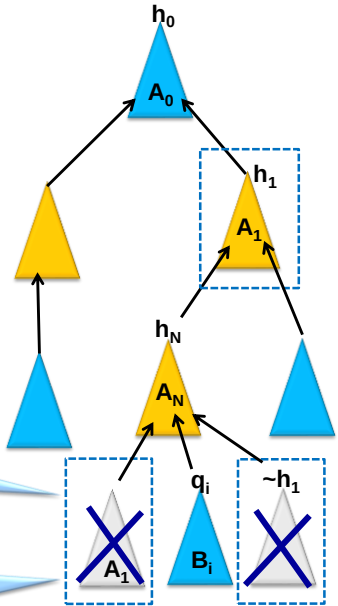
   Para cada $<B_i, q_i>$ tal que, la línea de argumentación $[<A_0,h_0>, <A_1,h_1>, ... <A_N, h_N>, <B_i, q_i>]$ es aceptable, existe un nodo hijo de $<A_N, h_N>$ etiquetado $<B_i, q_i>$



Definición de Árbol de dialéctica

Árbol de dialéctica para $<A_0, h_0>$:

1. La raíz del árbol es etiquetada con $<A_0, h_0>$
2. Dado un nodo cualquiera del árbol etiquetado $<A_N, h_N>$, donde $[<A_0,h_0>, <A_1,h_1>, ... <A_N, h_N>]$ es el camino desde la raíz hasta $<A_N, h_N>$.
   Sea $\{<B_1,q_1>,..., <B_k, q_k>\}$ el conjunto de todos los derrotadores de $<A_N, h_N>$.
   Para cada $<B_i, q_i>$ tal que, la línea de argumentación $[<A_0,h_0>, <A_1,h_1>, ... <A_N, h_N>, <B_i, q_i>]$ es aceptable, existe un nodo hijo de $<A_N, h_N>$ etiquetado $<B_i, q_i>$



No todo derrotador puede ser parte del árbol, ya que toda línea de argumentación debe ser aceptable.

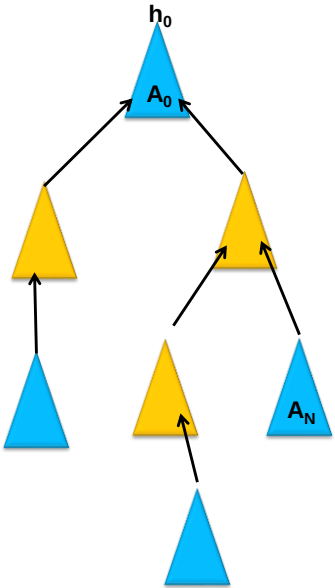
Por ejemplo: el argumento A_1 no puede estar como derrotador de A_N porque ya estaba en la misma línea de argumentación antes, y el otro argumento tampoco porque no es concordante con A_1 y ambos son de interferencia.

Definición de Árbol de dialéctica

Árbol de dialéctica para $<A_0, h_0>$:

1. La raíz del árbol es etiquetada con $<A_0, h_0>$
2. Dado un nodo cualquiera del árbol etiquetado $<A_N, h_N>$, donde $[<A_0,h_0>, <A_1,h_1>, ... <A_N, h_N>]$ es el camino desde la raíz hasta $<A_N, h_N>$.
   Sea $\{<B_1,q_1>,..., <B_k, q_k>\}$ el conjunto de todos los derrotadores de $<A_N, h_N>$.
   Para cada $<B_i, q_i>$ tal que, la línea de argumentación $[<A_0,h_0>, <A_1,h_1>, ... <A_N, h_N>, <B_i, q_i>]$ es aceptable, existe un nodo hijo de $<A_N, h_N>$ etiquetado $<B_i, q_i>$

   Si no existe ningún derrotador en tales condiciones entonces $<A_N, h_N>$ es una hoja.



Argumentos repetidos en un árbol de dialéctica

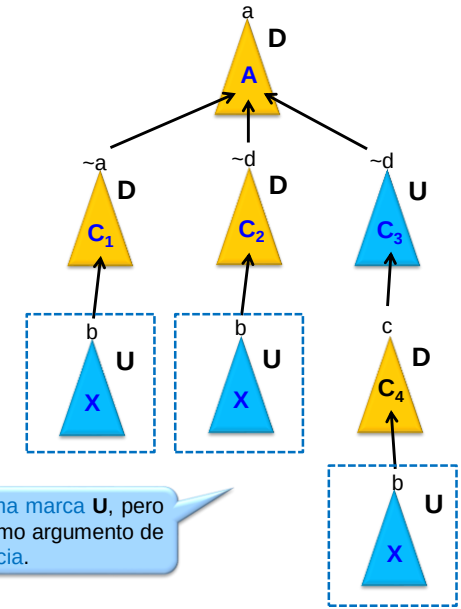
Aunque un argumento no puede repetirse en una línea de argumentación, podría repetirse en diferentes nodos del árbol.

```
(c -< ~b) > (~c -< x)
(b -< u) > (~b -< y)
```

$\Delta_1 =$ { a -< d. ; d -< x. ; ~d -< ~b. ; ~d -< ~c. ; ~b -< y. ; ~a -< ~b. ; ~c -< x. ; c -< ~b. ; b -< u. }

$\Pi_1 =$ { z. ; y. ; x. ; u. }

El argumento X aparece tres veces en este árbol de dialéctica porque derrota a tres argumentos diferentes.



En este ejemplo, X está siempre con la misma marca **U**, pero en diferentes niveles del árbol: dos veces como argumento de soporte, y una como argumento de interferencia.

Argumentos en diferentes partes de un árbol de dialéctica

Aunque un argumento no puede repetirse en una línea de argumentación, podría repetirse en diferentes nodos del árbol.

Este es otro PLR.

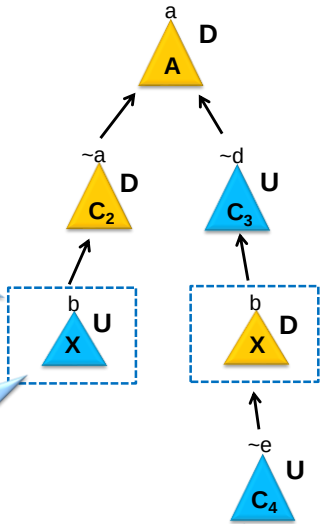
```
(c -< ~b) > (~c -< x)
(b -< e) > (~b -< y)
```

$\Delta_2 =$ { a -< d. ; d -< x. ; ~d -< ~b. ; ~b -< y. ; ~a -< ~b, e. ; b -< e. ; e -< u. ; ~e -< z. }

$\Pi_2 =$ { z. ; y. ; x. ; u. }

En este ejemplo, X aparece las dos veces como argumento de soporte para la raíz, pero con marcas diferentes: en un línea está **U**, y en la otra **D** porque tiene al derrotador C_4.

El argumento C_4 no es puede aparecer en la línea de argumentación [A, C_2, X] porque **C_4 y C_2 son contradictorios** (derivan e y ~e) y ambos de interferencia para la raíz.



La línea de argumentación [A, C_3, X, C_4] es aceptable, y por lo tanto, C_4 puede derrotar a X en esta línea.

Argumentos repetidos en diferentes líneas y con diferente rol y marca

Aunque un argumento no puede repetirse en una línea de argumentación, podría repetirse en diferentes nodos del árbol.

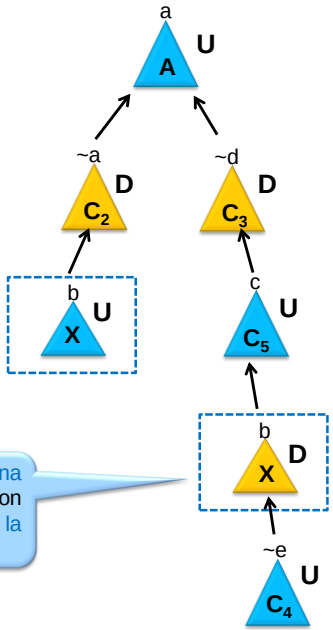
```
(c -< ~b) > (~c -< x)
(b -< e) > (~b -< y)
```

Es otro PLR diferente a los anteriores.

$\Delta_3 =$

```
a -< d.
d -< x.
~d -< ~c.
~a -< ~b, e.
~b -< y.
~c -< x.
c -< ~b.
b -< e.
e -< u.
~e -< z.
```

$\Pi_3 =$ { z. y. x. u. }



En este ejemplo, X aparece dos veces una vez de soporte y otra de interferencia, y con marcas diferentes: en un línea está **U**, y en la otra **D** porque tiene al derrotador C_4.

Argumentos y nodos

Aunque un argumento no puede repetirse en una línea de argumentación, podría repetirse en diferentes nodos del árbol.

```
(c -< ~b) > (~c -< x)
(b -< e) > (~b -< y)
```

Es otro PLR diferente a los anteriores.

$\Delta_3 =$

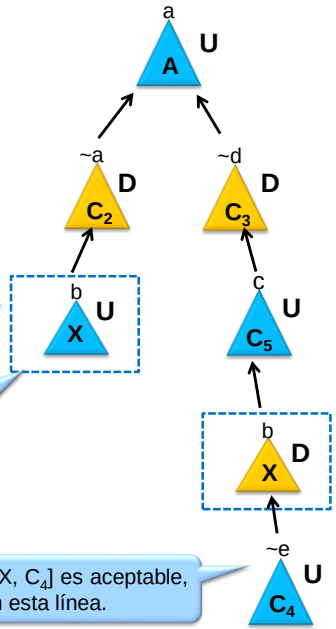
```
a -< d.
d -< x.
~d -< ~c.
~a -< ~b, e.
~b -< y.
~c -< x.
c -< ~b.
b -< e.
e -< u.
~e -< z.
```

$\Pi_3 =$

```
z.
y.
x.
u.
```

En este ejemplo, X aparece dos veces una vez de soporte y otra de interferencia, y con marcas diferentes: en un línea está **U**, y en la otra **D** porque tiene al derrotador C_4.

El argumento C_4 no es puede aparecer en la línea de argumentación [A, C_2, X] porque C_4 y C_2 son contradictorios (derivan e y ~e) y ambos de interferencia para la raíz.



La línea de argumentación [A, C_3, C_5, X, C_4] es aceptable, y por lo tanto, C_4 puede derrotar a X en esta línea.

Ejemplo integrador

Un nuevo ejemplo con el PLR (Δ_s, Π_s)



Δ_s

R1: rain –< cloudy

R2: ~rain –< dry_season

R3: yard_work –< grass_grown

R4: ~yard_work –< hire_gardener

- surf –< nice, spare_time
- nice –< waves
- ~nice –< rain
- spare_time –< ~busy
- ~busy –< ~working
- cold –< winter
- working –< monday
- busy –< yard_work
- many_surfers –< waves
- ~many_surfers –< monday

R2 >$_s$ R1
R4 >$_s$ R3

Prioridad entre reglas

Π_s

- ~working ← vacation
- few_surfers ← ~many_surfers
- ~surf ← ill
- monday
- cloudy
- dry_season
- waves
- grass_grown
- hire_gardener
- vacation

Un nuevo ejemplo con el PLR (Δ_s, Π_s)

Δ_s

- surf –< nice, spare_time
- nice –< waves
- ~nice –< rain
- R1: rain –< cloudy
- R2: ~rain –< dry_season
- spare_time –< ~busy
- ~busy –< ~working
- cold –< winter
- working –< monday
- busy –< yard_work
- R3: yard_work –< grass_grown
- R4: ~yard_work –< hire_gardener
- many_surfers –< waves
- ~many_surfers –< monday

R2 >$_s$ R1
R4 >$_s$ R3

Prioridad entre reglas

Π_s

- ~working ← vacation
- few_surfers ← ~many_surfers
- ~surf ← ill
- monday
- cloudy
- dry_season
- waves
- grass_grown
- hire_gardener
- vacation

surf

Busquemos una garantía para “surf” construyendo un árbol de dialéctica.

¿ surf ?



¿ surf o ~surf ?

Δ_s

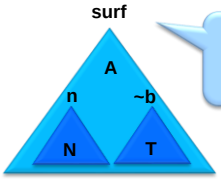
surf –< nice, spare_time
nice –< waves
~nice –< rain
R1 rain –< cloudy
R2 ~rain –< dry_season
spare_time –< ~busy
~busy –< ~working
cold –< winter
working –< monday
busy –< yard_work
R3 yard_work –< grass_grown
R4 ~yard_work –< hire_gardener
many_surfers –< waves
~many_surfers –< monday

R2 >$_s$ R1
R4 >$_s$ R3

Π_s

~working ← vacation
few_surfers ← ~many_surfers
~surf ← ill
monday
cloudy
dry_season
waves
grass_grown
hire_gardener
vacation

En el argumento que sustenta “surf”, se distinguen los sub-argumentos que sustentan: “nice” y “~busy”.



Busquemos una garantía para “surf” construyendo un árbol de dialéctica.

¿ surf ?



A partir de (Δ_s, Π_s) existe un argumento que sustenta “surf”

A = { surf –< nice, spare_time ; nice –< waves ; spare_time –< ~busy ; ~busy –< ~working }

N = { nice –< waves }

T = { ~busy –< ~working }

Construyendo la primera línea de argumentación

Δ_s

R1
surf –< nice, spare_time
nice –< waves
~nice –< rain
rain –< cloudy

R2
~rain –< dry_season
spare_time –< ~busy
~busy –< ~working
cold –< winter
working –< monday
busy –< yard_work

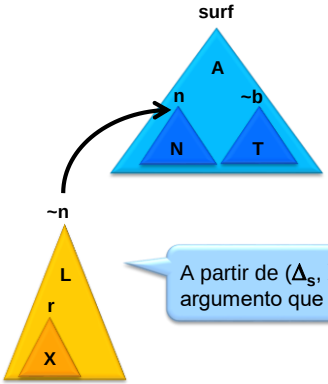
R3
yard_work –< grass_grown

R4
~yard_work –< hire_gardener
many_surfers –< waves
~many_surfers –< monday

$R2 >_s R1$
$R4 >_s R3$

Π_s

~working ← vacation
few_surfers ← ~many_surfers
~surf ← ill
monday
cloudy
dry_season
waves
grass_grown
hire_gardener
vacation



A partir de (Δ_s, Π_s) existe un argumento que sustenta “~nice”

El argumento <L,~nice> derrota por bloqueo a <A,surf> en el sub-argumento de desacuerdo <N, nice>,

¿ surf ?



A = { surf –< nice, spare_time ; nice –< waves ; spare_time –< ~busy ; ~busy –< ~working }

L = { ~nice –< rain ; rain –< cloudy }

Verificando que la línea de argumentación sea aceptable

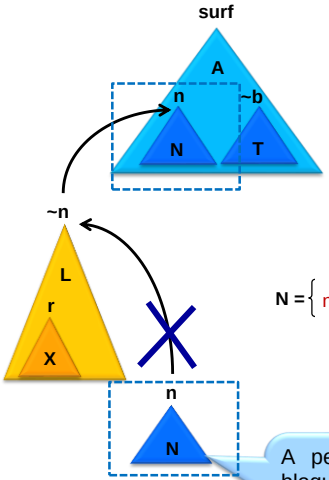
Δ_s

- surf –< nice, spare_time
- nice –< waves
- ~nice –< rain
- R1: rain –< cloudy
- R2: ~rain –< dry_season
- spare_time –< ~busy
- ~busy –< ~working
- cold –< winter
- working –< monday
- busy –< yard_work
- R3: yard_work –< grass_grown
- R4: ~yard_work –< hire_gardener
- many_surfers –< waves
- ~many_surfers –< monday

R2 >$_s$ R1
R4 >$_s$ R3

Π_s

- ~working ← vacation
- few_surfers ← ~many_surfers
- ~surf ← ill
- monday
- cloudy
- dry_season
- waves
- grass_grown
- hire_gardener
- vacation



¿ surf ?



A = { surf –< nice, spare_time ; nice –< waves ; spare_time –< ~busy ; ~busy –< ~working }

N = { nice –< waves }

L = { ~nice –< rain ; rain –< cloudy }

A pesar que el argumento <N,nice> derrota por bloqueo a <L,~nice> no puede agregarse al árbol de dialéctica porque no cumple con la condición 1 de línea de argumentación aceptable: es sub-argumento de descuerdo de un argumento anterior en la línea

Llegamos a una hoja

Δ_s

R1: surf –< nice, spare_time
nice –< waves
~nice –< rain
rain –< cloudy

R2: ~rain –< dry_season
spare_time –< ~busy
~busy –< ~working
cold –< winter
working –< monday
busy –< yard_work

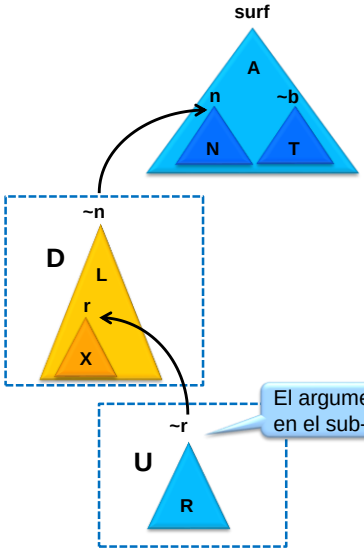
R3: yard_work –< grass_grown

R4: ~yard_work –< hire_gardener
many_surfers –< waves
~many_surfers –< monday

R2 >$_s$ R1
R4 >$_s$ R3

Π_s

~working ← vacation
few_surfers ← ~many_surfers
~surf ← ill
monday
cloudy
dry_season
waves
grass_grown
hire_gardener
vacation



¿ surf ?



A = { surf –< nice, spare_time ; nice –< waves ; spare_time –< ~busy ; ~busy –< ~working }

L = { ~nice –< rain ; rain –< cloudy } R1

El argumento <R, ~rain> es un derrotador propio de <L, ~nice> en el sub-argumento de desacuerdo <X, rain>

R = { ~rain –< dry_season } R2

Explorando otra línea de argumentación

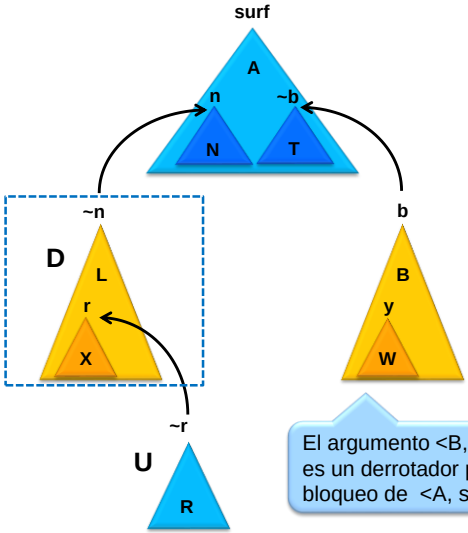
Δ_s

- surf –< nice, spare_time
- nice –< waves
- ~nice –< rain
- R1: rain –< cloudy
- R2: ~rain –< dry_season
- spare_time –< ~busy
- ~busy –< ~working
- cold –< winter
- working –< monday
- busy –< yard_work
- R3: yard_work –< grass_grown
- R4: ~yard_work –< hire_gardener
- many_surfers –< waves
- ~many_surfers –< monday

$R2 >_s R1$
$R4 >_s R3$

Π_s

- ~working ← vacation
- few_surfers ← ~many_surfers
- ~surf ← ill
- monday
- cloudy
- dry_season
- waves
- grass_grown
- hire_gardener
- vacation



El argumento <B, busy> es un derrotador por bloqueo de <A, surf>

¿ surf ?



A = { surf –< nice, spare_time; nice –< waves; spare_time –< ~busy; ~busy –< ~working }

L = { ~nice –< rain; rain –< cloudy }

B = { busy –< yard_work; yard_work –< grass_grown }

R = { ~rain –< dry_season }

Evitando una argumentación circular

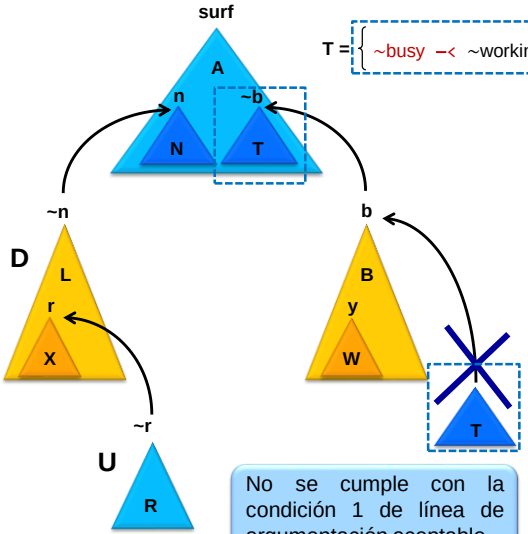
Δ_s

surf –< nice, spare_time
nice –< waves
~nice –< rain
R1 rain –< cloudy
R2 ~rain –< dry_season
spare_time –< ~busy
~busy –< ~working
cold –< winter
working –< monday
busy –< yard_work
R3 yard_work –< grass_grown
R4 ~yard_work –< hire_gardener
many_surfers –< waves
~many_surfers –< monday

R2 >$_s$ R1
R4 >$_s$ R3

Π_s

~working ← vacation
few_surfers ← ~many_surfers
~surf ← ill
monday
cloudy
dry_season
waves
grass_grown
hire_gardener
vacation



T = { ~busy –< ~working }

¿ surf ?



A = { surf –< nice, spare_time
nice –< waves
spare_time –< ~busy
~busy –< ~working }

L = { ~nice –< rain
rain –< cloudy }

B = { busy –< yard_work
yard_work –< grass_grown }

R = { ~rain –< dry_season }

No se cumple con la condición 1 de línea de argumentación aceptable.

Marcado completo del árbol

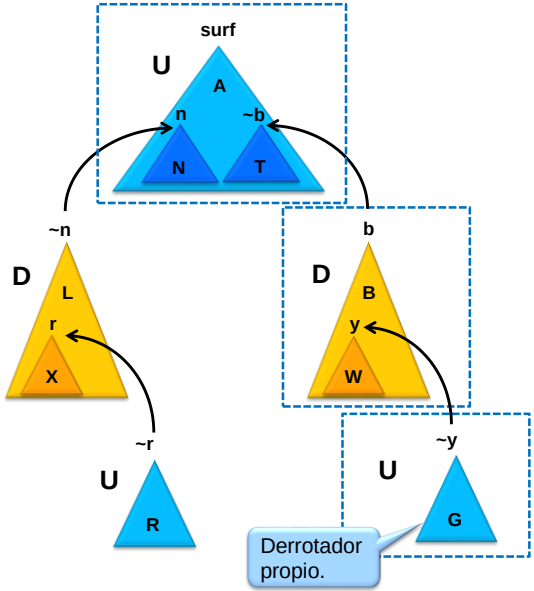
Δ_s

- surf –< nice, spare_time
- nice –< waves
- ~nice –< rain
- R1: rain –< cloudy
- R2: ~rain –< dry_season
- spare_time –< ~busy
- ~busy –< ~working
- cold –< winter
- working –< monday
- busy –< yard_work
- R3: yard_work –< grass_grown
- R4: ~yard_work –< hire_gardener
- many_surfers –< waves
- ~many_surfers –< monday

R2 >$_s$ R1
R4 >$_s$ R3

Π_s

- ~working ← vacation
- few_surfers ← ~many_surfers
- ~surf ← ill
- monday
- cloudy
- dry_season
- waves
- grass_grown
- hire_gardener
- vacation



¿ surf ?



A = { surf –< nice, spare_time ; nice –< waves ; spare_time –< ~busy ; ~busy –< ~working }

L = { ~nice –< rain ; rain –< cloudy }

B = { busy –< yard_work ; yard_work –< grass_grown }

R3

R = { ~rain –< dry_season }

R4

G = { ~yard_work –< hire_gardener }

Respuesta: sí, disfrutemos de un día de surf

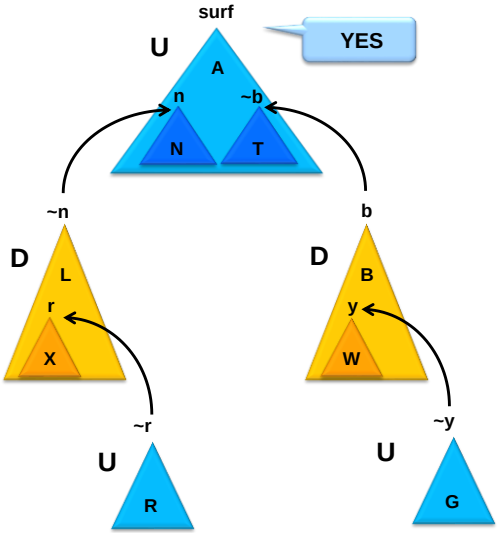
Δ_s

surf –< nice, spare_time
nice –< waves
~nice –< rain
R1 rain –< cloudy
R2 ~rain –< dry_season
spare_time –< ~busy
~busy –< ~working
cold –< winter
working –< monday
busy –< yard_work
R3 yard_work –< grass_grown
R4 ~yard_work –< hire_gardener
many_surfers –< waves
~many_surfers –< monday

$R2 >_s R1$
$R4 >_s R3$

Π_s

~working ← vacation
few_surfers ← ~many_surfers
~surf ← ill
monday
cloudy
dry_season
waves
grass_grown
hire_gardener
vacation



¿ surf ?



A = { surf –< nice, spare_time
nice –< waves
spare_time –< ~busy
~busy –< ~working }

L = { ~nice –< rain
rain –< cloudy }

B = { busy –< yard_work
yard_work –< grass_grown }

R = { ~rain –< dry_season }

G = { ~yard_work –< hire_gardener }

Respuesta: sí, disfrutemos de un día de surf

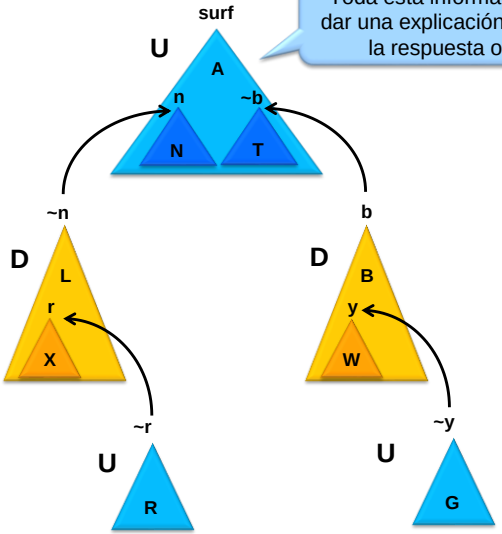
Δ_s

surf –< nice, spare_time
nice –< waves
~nice –< rain
R1 rain –< cloudy
R2 ~rain –< dry_season
spare_time –< ~busy
~busy –< ~working
cold –< winter
working –< monday
busy –< yard_work
R3 yard_work –< grass_grown
R4 ~yard_work –< hire_gardener
many_surfers –< waves
~many_surfers –< monday

$R2 >_s R1$
$R4 >_s R3$

Π_s

~working ← vacation
few_surfers ← ~many_surfers
~surf ← ill
monday
cloudy
dry_season
waves
grass_grown
hire_gardener
vacation



Toda esta información permite dar una explicación detallada de la respuesta obtenida.



A = { surf –< nice, spare_time ; nice –< waves ; spare_time –< ~busy ; ~busy –< ~working }

L = { ~nice –< rain ; rain –< cloudy }

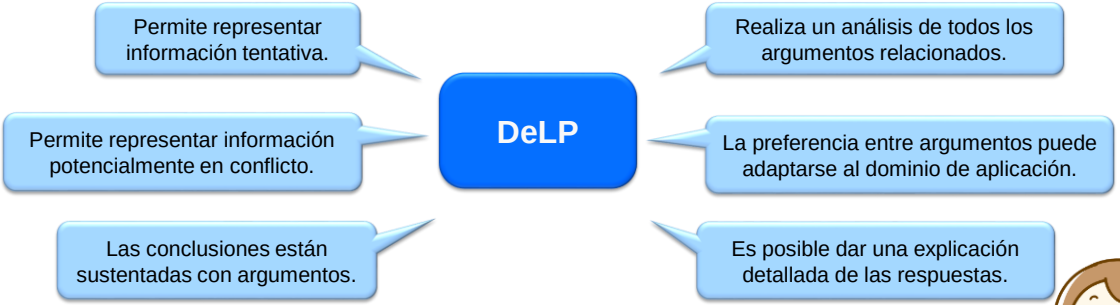
B = { busy –< yard_work ; yard_work –< grass_grown }

R = { ~rain –< dry_season }

G = { ~yard_work –< hire_gardener }

Comentarios finales

Comentarios finales





Referencias:

[1] **Programación Lógica Rebatible.** Alejandro J. García. (Capítulo 2) Tesis de Doctor en Ciencias de la Computación. (2000) http://www.cs.uns.edu.ar/~ajg/papers/AGarciaPhD.pdf

[2] **Defeasible logic programming: An argumentative approach.** Alejandro. J. García and Guillermo R. Simari. Journal of Theory and Practice of Logic Programming. (2004) DOI: https://doi.org/10.1017/S1471068403001674

[3]. **Defeasible Logic Programming: DeLP-servers, Contextual Queries, and Explanations for Answers**. A.García & G.Simari. Journal of Argument & Computation, (2014), https://content.iospress.com/articles/argument-and-computation/869767



Muchas gracias por la atención!



Videos complementarios



Como en las dos partes anteriores, lo que sigue de este material en formato “de lectura”, se complementa con el video asociado al enlace de la siguiente imagen.

